

课程笔记

C 标准库原理

南京大学 操作系统原理 2026 春季学期 第 09 讲

lecture-to-notes

2026 年 4 月 5 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 25 分钟

视频链接: <https://www.bilibili.com/video/BV1GHXCBkEZf>

目录

1 Overview: C 语言的三层封装	2
1.1 C 的完整能力: FFI, ABI, Inline Assembly	2
1.2 Inline Assembly 解密	3
1.3 构建应用生态: 组合、复用、分层	4
1.4 本章小结	4
2 指令集和计算的封装	5
2.1 printf 家族: 共享实现	5
2.2 setjmp / longjmp: 非局部跳转	6
2.3 本章小结	6
3 系统调用的封装	7
3.1 进程管理	7
3.2 本章小结	7
4 环境的封装	8
4.1 environ: 环境变量	8
4.2 __libc_start_main: main 之前的世界	9
4.3 本章小结	9
5 总结与延伸	10
5.1 讲者的核心信息	10
5.2 结构化提炼	10
5.3 与前几讲的关联	10
5.4 拓展阅读	10

1 Overview: C 语言的三层封装

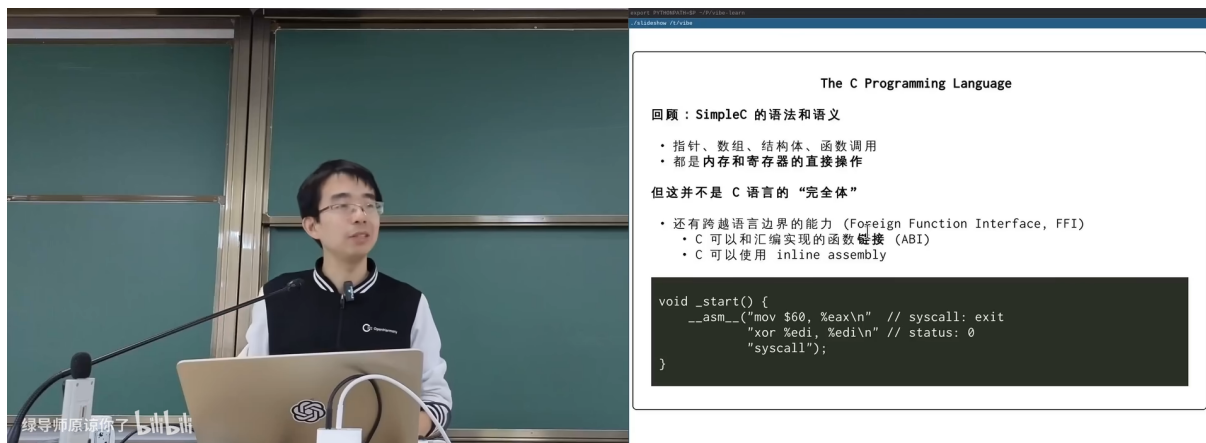


图 1: The C Programming Language: SimpleC 的语法和语义 vs. 完整 C 语言的能力¹

jyy 提出本讲的核心结构: C 标准库是操作系统和应用程序之间的三层封装:

C 标准库的三层封装

1. **指令集和计算的封装**: 把 CPU 指令 (包括 SIMD、浮点、原子操作) 封装为可移植的 C 函数 (memcpy, sqrt, printf)
2. **系统调用的封装**: 把操作系统的 syscall 封装为标准接口 (fopen, malloc, fork)
3. **环境的封装**: 管理进程启动和退出 (`_start` $\rightarrow$ `main` $\rightarrow$ `exit`), 包括环境变量、命令行参数、`atexit` 注册

1.1 C 的完整能力: FFI, ABI, Inline Assembly

C 不止是 SimpleC

- **FFI** (Foreign Function Interface): 调用其他语言的函数
- **ABI** (Application Binary Interface): 与编译器/链接器约定的函数调用规范
- **Inline Assembly**: 在 C 代码中直接嵌入汇编指令

¹视频画面时间区间: 00:10:00–00:10:30。帧实际内容: slide “The C Programming Language”——回顾 SimpleC (指针、数组、结构体、函数调用、相当于内存和寄存器的直接操作), 但这并不是 C 语言的“完全体”。C 还有 FFI (Foreign Function Interface)、可以和汇编器约定的函数碎片 (ABI)、可以直接使用 inline assembly。附 `_start()` + `syscall` 汇编代码。

1.2 Inline Assembly 解密

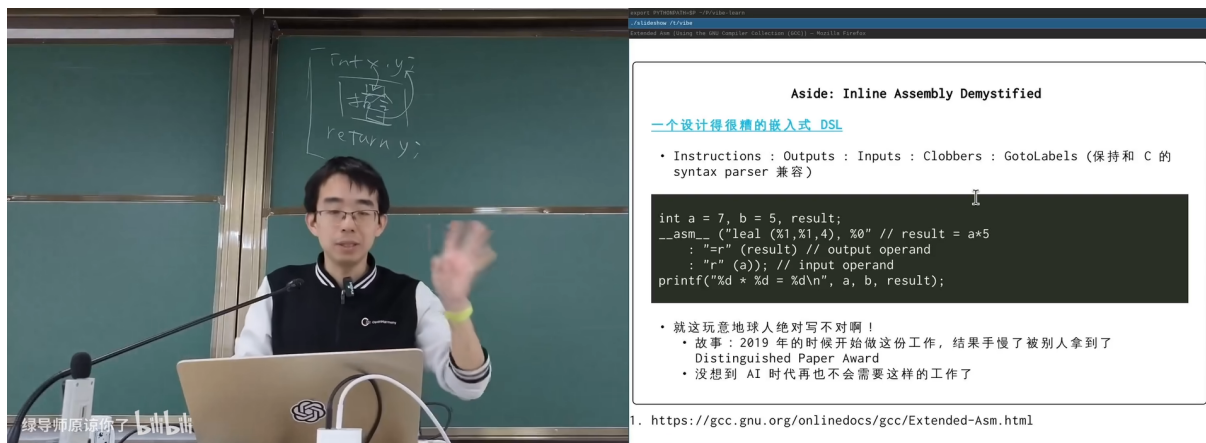


图 2: Inline Assembly Demystified: GCC Extended Asm 的四个组成部分²

```
1 int a = 7, b = 5, result;
2 __asm__ ("add %1, %0"           // Instructions
3       : "=r" (result)          // Output operand
4       : "r" (a));              // Input operand
5 printf("%d + %d = %d\\n", a, b, result);
```

Listing 1: GCC Extended Asm 语法

Inline Assembly 的风险

jyy 提到：“就是从想把硬件特别放到脑子里用”——2019 年的时候开始做这份工作，结果半年没得到一篇 Distinguished Paper Award。“现在 AI 时代的代码也不会需要这样的工作了？”——但理解 inline asm 仍然是理解 C 标准库实现的前提。

²视频画面时间区间：00:18:43–00:19:13。帧实际内容：slide “Aside: Inline Assembly Demystified”——一个比目前最酷的嵌入式 ISA，Instructions / Outputs / Inputs / Clobbers / GotoLabels（保持和 C 的 syntax parser 兼容）。示例：int a=7, b=5, result; __asm__("add %1, %0" : "=r"(result) : "r"(a));。提到 2019 年的 Distinguished Paper Award。

1.3 构建应用生态：组合、复用、分层

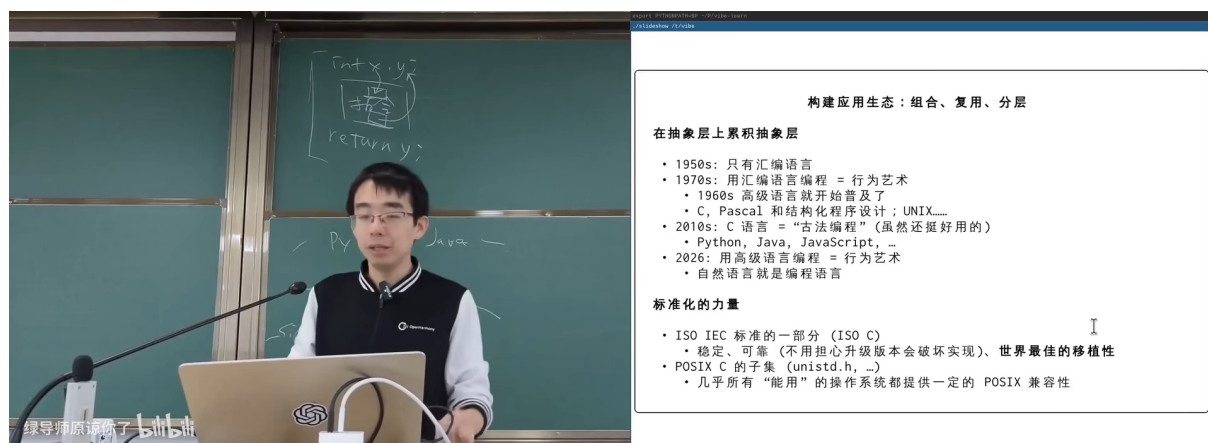


图 3: 抽象层的历史演化：从 1950s 汇编到 2020s 自然语言编程³

标准化的力量

- **ISO C**: C 语言国际标准，定义了语言核心和标准库
- **POSIX**: 操作系统接口标准 (unistd.h)，几乎所有“像样的”操作系统都保持兼容
- 标准化让**代码可以跨平台移植**——写一次，到处编译

1.4 本章小结

C 标准库 = 指令集封装 + 系统调用封装 + 环境封装。ISO C 和 POSIX 标准化让这套封装跨平台可移植。理解 C 标准库就是理解“应用程序和操作系统之间的翻译层”。

³视频画面时间区间：00:28:43-00:29:13。帧实际内容：slide “构建应用生态：组合、复用、分层”——在抽象层上累积抽象层。1950s 只有汇编器 → 1960s 高级语言出现（行为艺术）→ 1960s 高级语言开始普及 → C/Pascal 用纯代码开发（UNIX 奠定性作用的）→ Python/-Java/JavaScript → 2020s 自然语言就能编程（6 万字文档）。标准化的力量：ISO IEC 标准的一部分 (ISO C)、标准 C 可以帮助解决公共模块完全标准化的问题、POSIX C 的子集 (unistd.h)、几乎所有“像样的”操作系统都保持 POSIX 兼容性。

2 指令集和计算的封装

2.1 printf 家族：共享实现

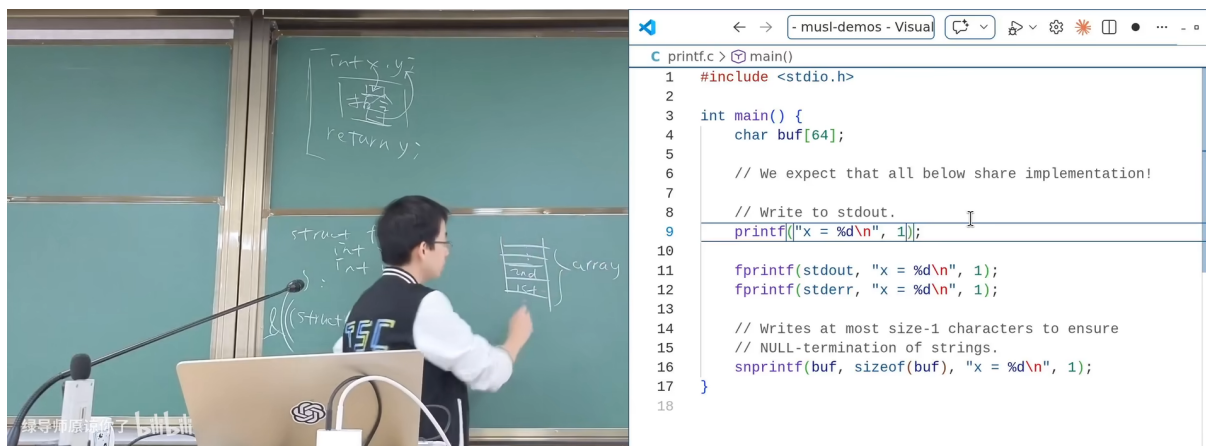


图 4: printf.c: printf / fprintf / snprintf 共享同一个底层实现⁴

```
1 // Write to stdout
2 printf("x=%d\n", 1);
3
4 // Write to specific FILE*
5 fprintf(stdout, "x=%d\n", 1);
6 fprintf(stderr, "x=%d\n", 1);
7
8 // Write to buffer (safe, with size limit)
9 snprintf(buf, sizeof(buf), "x=%d\n", 1);
```

Listing 2: printf 家族的统一接口

printf 的本质

printf 家族的核心是一个**格式化引擎**——把格式字符串 + 可变参数转换为字节序列。输出目标不同（stdout、FILE*、buffer），但格式化逻辑完全共享。

这就是 C 标准库“封装”的典型模式：一个**核心实现** + 多个**前端接口**。

⁴视频画面时间区间：00:44:00–00:44:30。帧实际内容：VS Code 显示 printf.c 源码，int main() 中展示 printf("x = %d", 1)、fprintf(stdout, "x = %d", 1)、fprintf(stderr, "x = %d", 1) 和 snprintf(buf, sizeof(buf), "x = %d", 1) 共享相同的格式化引擎。

2.2 setjmp / longjmp: 非局部跳转

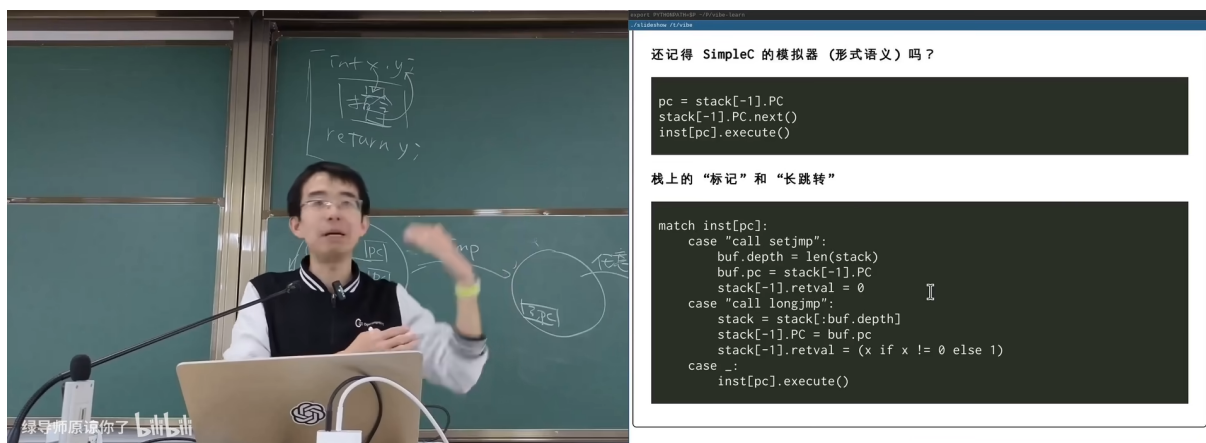


图 5: setjmp/longjmp 的 SimpleC 模拟器实现：“标记”和“长跳转”⁵

setjmp/longjmp

setjmp 保存当前执行上下文（寄存器、栈指针、PC）到 **jmp_buf**。**longjmp** 从任意深度的函数调用中“跳回”到 **setjmp** 的位置。

用状态机视角理解：

- **setjmp**：“拍快照”——记录当前栈深度和 PC
- **longjmp**：“回滚”——恢复栈到快照深度，设置返回值，跳到快照 PC

这是 C 语言中最接近“异常处理”的机制——**try/catch** 在 C++ 中本质上就是 **setjmp/longjmp** 的封装。

2.3 本章小结

C 标准库把 CPU 指令封装为可移植函数：**printf** 家族共享格式化引擎、**string.h** 封装内存操作、**setjmp/longjmp** 实现非局部跳转。核心设计模式：一个底层实现 + 多个前端接口。

⁵视频画面时间区间：00:51:25-00:51:55。帧实际内容：slide “还记得 SimpleC 的模拟器（形式语义）吗？”——展示 **setjmp/longjmp** 在 SimpleC 状态机中的实现。**setjmp** = 保存 **stack depth** 和 **PC** 到 **buf**；**longjmp** = 恢复 **stack** 到 **buf.depth**，设置 **retval**，跳转到 **buf.pc**。match **inst[pc]**: case “call setjmp” → **buf.depth** = **len(stack)**; case “call longjmp” → **stack** = **stack[:buf.depth]**, **retval** = (**x** if **x** != 0 else 1)。

3 系统调用的封装

3.1 进程管理

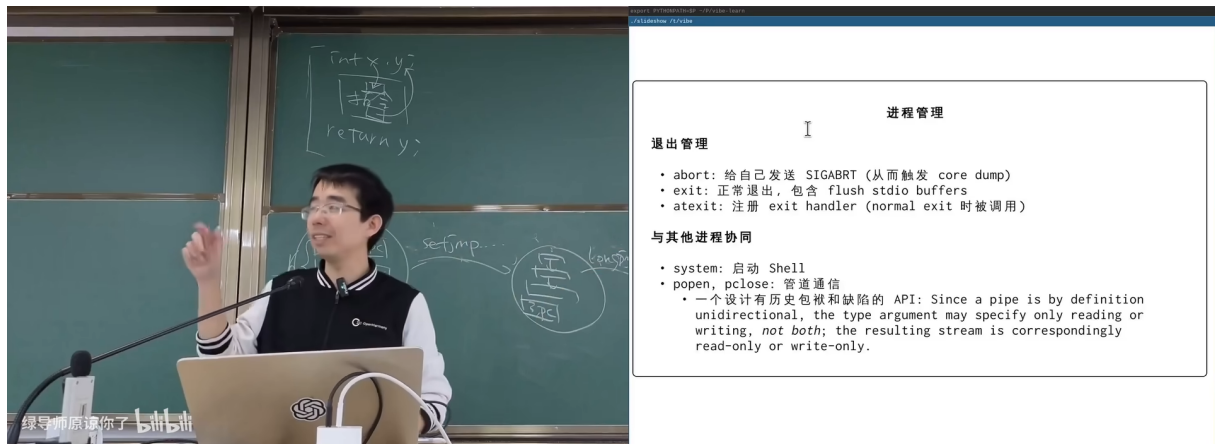


图 6: C 标准库的进程管理封装: abort / exit / atexit / system / popen⁶

C 标准库的进程管理 API

退出管理:

- abort(): 给自己发送 SIGABRT (从而做 core dump)
- exit(): 正常退出, 包含 flush stdio buffers
- atexit(): 注册 exit handler (normal exit 时被调用)

与其他进程交互:

- system(): 启动 Shell 执行命令
- popen()/pclose(): 创建管道与子进程通信

popen 的设计缺陷

jyy 吐槽: “一个设计充满着老旧味道的 API”——popen 的管道是**单向的** (“Since a pipe is by definition unidirectional, the type argument may specify only reading, or writing, not both”). 想要双向通信? 得用两个 popen 或者直接用 pipe + fork。

3.2 本章小结

C 标准库把操作系统的 syscall 封装为标准接口: 文件操作 (fopen/fread/fwrite)、进程管理 (exit/system/popen)、内存管理 (malloc/free)。封装的价值: 跨平台可移植 + 隐藏 syscall 细节。

⁶视频画面时间区间: 01:02:46–01:03:16。帧实际内容: slide “进程管理”——退出管理: abort (给自己发送 SIGABRT, 从而数据 core dump)、exit (正常退出, 包含 flush stdio buffers)、atexit (注册 exit handler, normal exit 时被调用)。与其他进程的互动: system (启动 Shell)、popen/pclose (管道通信) ——“一个设计充满着老旧味道的 API: Since a pipe is by definition unidirectional...”。

4 环境的封装

4.1 environ: 环境变量

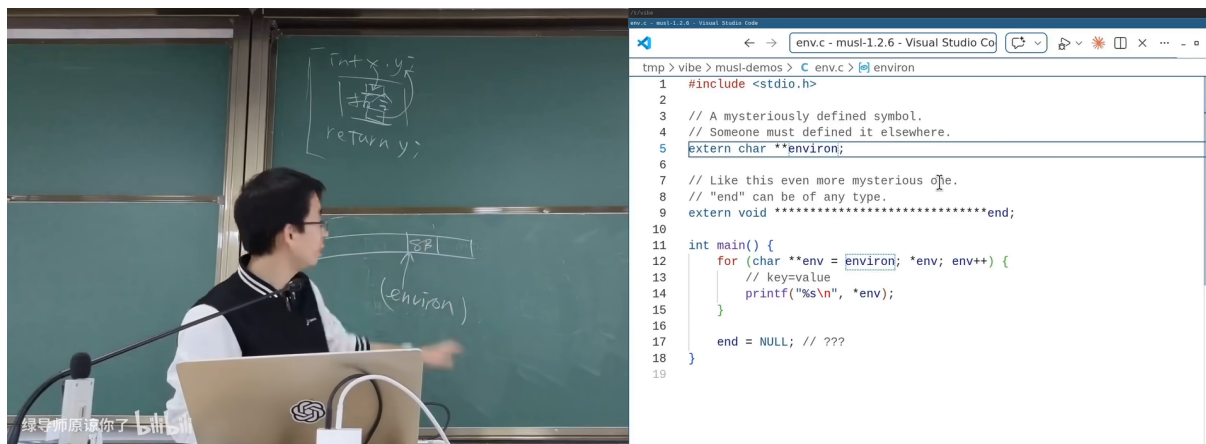


图 7: environ: 环境变量的 C 接口——__environ 指针和 extern void *end⁷

```
1 #include <stdio.h>
2
3 extern char **environ; // Someone must define it elsewhere
4
5 int main() {
6     for (char **env = environ; *env; env++) {
7         printf("%s\n", *env); // key=value
8     }
9 }
```

Listing 3: 遍历环境变量

environ 的“神秘”来源

extern char **environ 是一个“神秘地定义在别处”的符号。它不是你定义的，不是库定义的——而是 C 运行时 (CRT) 在 __libc_start_main 中设置的。execve 的第三个参数 envp 被传递给 CRT，CRT 把它存到全局变量 environ 中。

⁷ 视频画面时间区间: 01:16:35–01:17:05。帧实际内容: VS Code 显示 environ 代码——#include <stdio.h>, extern char **environ; // A mysteriously defined symbol, Someone must defined it elsewhere. 遍历 environ 打印所有 key=value 对。以及 extern void *end; // “end” can be of any type, end = NULL; // ???。

4.2 __libc_start_main: main 之前的世界

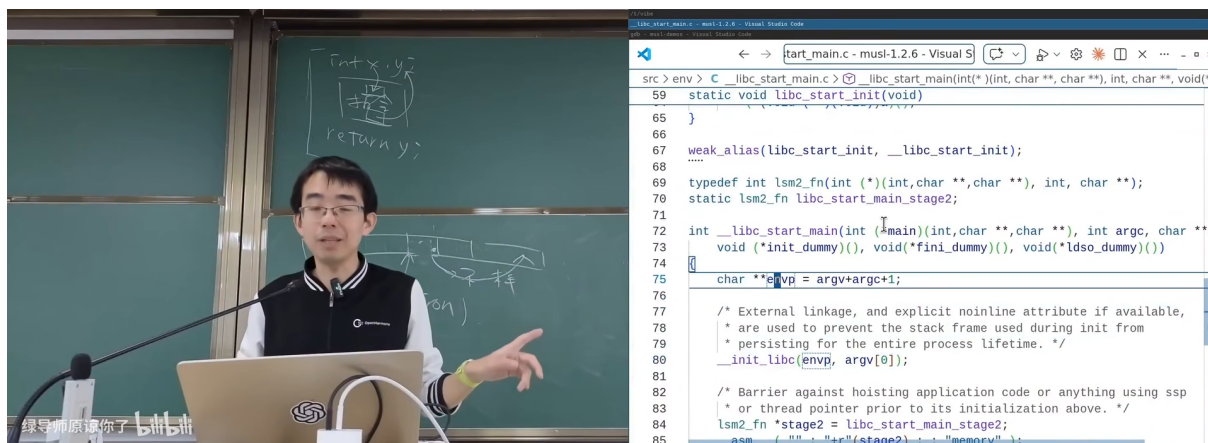
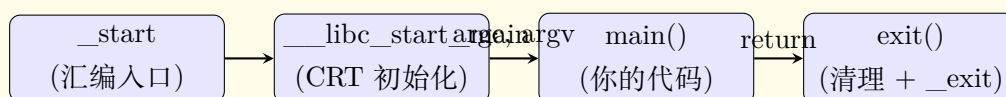


图 8: musl libc 的 `__libc_start_main` 源码：从 `_start` 到 `main` 的完整链路⁸

从 `_start` 到 `main` 的完整链路



`__libc_start_main` 做了什么：

1. `__init_libc`: 初始化 libc 内部状态
2. 从栈上提取 `argc`, `argv`, `envp`
3. 设置 `environ = argv + argc + 1`
4. 调用 `main(argc, argv)`
5. 调用 `exit(main 的返回值)`

4.3 本章小结

C 标准库的“环境封装”管理了 `main` 之前和之后的一切：`_start` → `__libc_start_main` → `main` → `exit`。环境变量通过 `execve` 传递，CRT 负责初始化。你的 `main()` 只是这个完整生命周期中的一小段。

⁸视频画面时间区间：01:21:35–01:22:05。帧实际内容：VS Code 显示 musl libc 的 `__libc_start_main` 实现——`int __libc_start_main(int (*main)(), ..., int argc, char **argv, char **envp)` 中调用 `__init_libc`, `__libc_start_init`, 然后 `char **p = argv+argc+1; // environ`, 最后调用 `exit(main(argc, argv))`。

5 总结与延伸

5.1 讲者的核心信息

$$\text{C 标准库} = \underbrace{\text{指令集封装}}_{\text{printf, memcpy, setjmp}} + \underbrace{\text{syscall 封装}}_{\text{fopen, malloc, exit}} + \underbrace{\text{环境封装}}_{\text{__start, environ, atexit}}$$

5.2 结构化提炼

四个核心 Takeaway

1. **C 标准库是三层封装**——指令集（计算能力）、系统调用（OS 能力）、环境（生命周期管理）。理解这三层就理解了“应用程序如何与操作系统交互”。
2. **“一个实现 + 多个前端”**——printf 家族共享格式化引擎，FILE* 封装 fd + 缓冲区。这是 C 标准库的核心设计模式。
3. **main 不是程序的起点**——`\_\_start` → `\_\_libc_start_main` → `main` → `exit`。CRT 在 `main` 之前初始化了 `environ`、设置了 `atexit`、准备了 `argc/argv`。
4. **标准化 = 可移植性**——ISO C + POSIX 让同一份代码在 Linux/macOS/Windows 上编译运行。这是 C 语言 50 年后仍然是系统编程首选的根本原因。

5.3 与前几讲的关联

- 第 02 讲说“strace 追踪 syscall” → 第 09 讲解释了 C 标准库如何把 syscall 封装成 `fopen/malloc`
- 第 05 讲说“fork + execve” → 第 09 讲解释了 `execve` 如何传递 `environ` 给 `\_\_libc_start_main`
- 第 07 讲说“fd 和 pipe” → 第 09 讲解释了 FILE* 如何在 fd 之上加缓冲区

5.4 拓展阅读

- musl libc 源码: <https://musl.libc.org/>
- GCC Extended Asm: <https://gcc.gnu.org/onlinedocs/gcc/Extended-Asm.html>
- jyy Wiki: <https://jyywiki.cn/>